

Object-Oriented Programming & Design Laboratory Work 2

10 points

Instructor: Izbassar Assylzhan

Referenced from: Pakizar Shamoï

Deadline: 8th week, on your practice time. No late submissions will be accepted! Please, read carefully class requirements. Remember about naming conventions. Keep your classes separately.

The lab work requires much more effort & time, rather than practices. For that reason, please, plan your time carefully! Good luck! :)

Introduction

This lab is designed to improve your OOP coding skills and deepen your understanding of key OOP concepts of inheritance, polymorphism, and abstract classes. Please carefully follow the guidelines below.

Key Rules and Guidelines

1. **Prohibition on AI-generated Code.** The use of AI-generated code (e.g., ChatGPT or other similar tools) is strictly prohibited. If AI-generated code is found in your lab, even in a single problem, you will receive 0 points for the entire lab. You are expected to write and develop the code on your own.
2. Every task costs equally **2 points**, and totally 10 points.

Note: Code hints are given to you so you can have an easy start and initial structure. For that reason, the lab work seems big.

Final Note

"This lab is for your growth as a developer. Take it seriously, put in your best effort, and most importantly, **learn from the process**. Success doesn't come from shortcuts but from persistence and practice. Be persistent, please, even when the challenges seem difficult. Every problem you solve is a step towards becoming a better software engineer.", - Pakizar Shamoï.

Problem 1

The current task consist 3 parts, as written below.

Part a: 3D Shapes

Create an abstract class for 3D shapes with methods `volume()` and `surfaceArea()`. Then, create data types `Cylinder`, `Sphere`, and `Cube` that extend this class.

Code snippet:

```
1 public abstract class Shape3D {
2     public abstract double volume();
3     ...
4     // continue
5 }
6
7 public class Cylinder extends Shape3D {
8     private double radius, height;
9     public Cylinder(double r, double h) {
10         this.radius = r;
11         this.height = h;
12     }
13     @Override
14     public double volume() {
15         return Math.PI * radius * radius * height;
16     }
17     ...
18     // continue
19 }
20 // continue with other classes
```

Part b: Library System

Design a system for managing a library's collection. You have different types of items, such as books, DVDs, and magazines. Each item has some common properties (e.g., title, author, and publication year), but also specific properties. Create an abstract class `LibraryItem` and one of the subclasses (choose one!) such as `Book`, `DVD`, and `Magazine`, inheriting from `LibraryItem`. implement the specific properties and methods for the subclass you chose.

Code snippet:

```
1 public abstract class LibraryItem {
2     private String title;
3     private String author;
4     private int publicationYear;
5     // constructor, getters, and setters
6     // toString() and other methods
7 }
8 public class Book extends LibraryItem {
9     private int numberOfPages;
10    public Book(String title, String author, int year, int pages) {
11        super(title, author, year);
12        this.numberOfPages = pages;
13    }
14    // continue with other necessary codes
15 }
```

Part c: Superclass and Subclass

Create a superclass and subclass of your choice. Override `equals()` and `hashCode()` methods in both classes. Test the classes by adding objects to a `HashSet` and verifying no duplicate entries. **This is an example, you have to think of another example.**

Code snippet:

```
1 // Superclass definition
2 public class Person {
3     private String name;
4     private int age;
5     /* write down:
6     / getters, setters,
7     / toString(),
8     / equals(), hashCode()
9     */
10 }
11 // Subclass definition
12 public class Employee extends Person {
13     private String employeeId;
14     // Override equals() and hashCode() methods
15     @Override
16     public boolean equals(Object o) {
17         // Custom equality check
18     }
19     @Override
20     public int hashCode() {
21         // Custom hash code logic
22     }
23 } // don't forget test classes
```

Problem 2: Chess Pieces

Create a data type for chess pieces. Inherit from the abstract class `Piece`, and then create subclasses `Rook`, `King`, `Queen`, `Bishop`, `Knight`, and `Pawn`. Implement the `isLegalMove()` method to check valid moves that determines whether the given piece can move from `Position a` to `b`. Yes, `Position` is a class. Use it if you can't play chess - Chess Wikipedia Page.

Code snippet:

```
1 public abstract class Piece {
2     Position a;
3     public abstract boolean isLegalMove(Position b);
4     // anything else you need, continue
5 }
6 public class Rook extends Piece {
7     @Override
8     public boolean isLegalMove(Position b) {
9         // Check horizontal or vertical moves
10    }
11    // anything else you need, continue
```

```
12 //continue with other classes
13 }
```

Bonus: Chess Game Implementation

Create a class Board and test class to manage the current state of a chess game and fully imitate the game. Implement methods to handle user input, draw the board on the console, draw the board on a console, check for illegal moves, etc.

The cost of the bonus is from 1 to 4 points depending on your result.

Problem 3: Bank Account System

Write the Account class and use it as a base class, then write two derived classes called SavingsAccount and CheckingAccount.

SavingsAccount (aka "deposit"):

- In addition to the attributes of an Account object, it should have an interest rate variable (%) and a method to add interest to the account.

CheckingAccount (there is a charge for each transaction):

- In addition to the attributes of an Account object, it should have a counter variable to store the number of transactions done by the user.
- Define FREE_TRANSACTIONS as the number of free transactions. Also, include a method deductFee() that withdraws \$0.02 for each transaction (withdrawal or deposit) exceeding the free transactions.

Ensure that you override the necessary methods of the Account class in both derived classes.

Code snippet:

```
1 public class Account{
2     private double balance; // The current balance
3     private int accNumber; // The account number
4     public Account(int a){
5         balance = 0.0;
6         accNumber = a;
7     }
8     public void deposit(double sum) { , , , }
9     public void withdraw(double sum) { , , , }
10    public double getBalance() { , , , }
11    public double getAccountNumber() { , , , }
12    public void transfer(double amount, Account other) { , , , }
13    public String toString() { , , , }
14    public final void print() {
15        // Do not override this method, override the toString method
16        System.out.println(toString());
17    }
18 }
```

Bank Class

Create a Bank class, which contains a Vector of Account objects.

- The accounts in the Vector can be instances of Account, SavingsAccount, or CheckingAccount.
- Write an `update(...)` method that iterates through each account and processes transactions:
 - SavingsAccount objects get interest added.
 - CheckingAccount objects get fees deducted.
- Create test accounts (some of each type).
- Implement methods for opening and closing accounts.

Problem 4: Electrical Circuit System

When electricity moves through a wire, it is subject to electrical friction or *resistance*. When a resistor with resistance R is connected across a potential difference V , Ohm's law asserts that it draws current $I = V/R$ and dissipates power V^2/R . A network of resistors connected across a potential difference behaves as a single resistor, which we call the *equivalent resistance*.

You should have an abstract superclass `Circuit` that encapsulates the basic properties of a resistor network. For example, each network has a method `getResistance()` that returns the equivalent resistance of the circuit.

Code snippet:

```
1 public abstract class Circuit {
2     public abstract double getResistance();
3     public abstract double getPotentialDiff();
4     public abstract void applyPotentialDiff(double V);
5     public double getPower() {
6         // your code
7     }
8     public double getCurrent() {
9         // your code
10    }
11 }
```

A series-parallel resistor network is either (i) a single resistor or (ii) built up by connecting two resistor networks in series or parallel. So create three `Circuit` class subclasses: `Resistor`, `Series`, and `Parallel`.

Class Resistor:

- The class `Resistor` contains a constructor that sets the resistance in Ohms and an accessor method to return it.
- It also has a private field `potentialDifference()` with `get/set` methods.

Class Series:

- The class `Series` contains a constructor that takes two circuit objects as inputs and represents a circuit with the two components in series.

Class Parallel:

- The class `Parallel` is similar to `Series`, but it uses the reciprocal rule instead of the additive rule to compute the equivalent resistance.

Your goal is to be able to compose circuits as in the following code fragment, which represents the circuit depicted below.

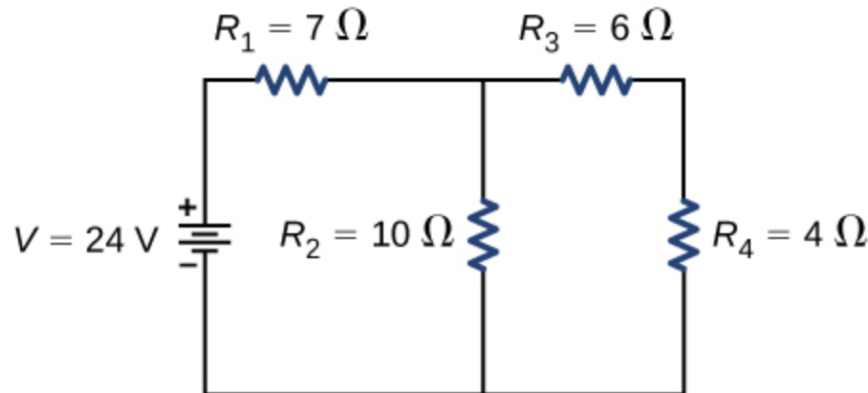


Figure 1: Example circuit referenced from Resistors in Series and Parallel — OpenStax University Physics II

Code snippet:

```
1 class CircuitTest {
2     public static void main(String[] args) {
3         Circuit r1 = new Resistor(7.0);
4         Circuit r2 = new Resistor(10.0);
5         Circuit r3 = new Resistor(6.0);
6         Circuit r4 = new Resistor(4.0);
7         Circuit c1 = new Parallel(r1, r2);
8         Circuit c2 = new Series(r3, r4);
9         Circuit circuit = new Parallel(c1, c2);
10        double R = circuit.getResistance();
11    }
12 }
```

Notes:

- The potential difference across each section depends on whether the circuit is series or parallel.
- For a parallel circuit, the potential difference across each branch is equal to the potential difference across the whole parallel circuit.

- For a series circuit, first find the current I from Ohm's law $I = V/R$, where V is the potential difference across the series circuit and R is its total resistance. The potential difference across each resistor is equal to the current times the resistance of that resistor.

Problem 5: Pet Management System

You are tasked with creating a **Pet Management System** that models different types of people (Employee, Student, PhDStudent) and pets. In this system, people may or may not own pets, which can be various animals like cats, dogs, or birds. In addition, sometimes people need to leave their pets with others in the system, when they go on vacation.

Code snippet:

```

1 public class PetTest {
2     public static void main(String[] args) {
3         Person john = new Employee("John", 30, "Engineer");
4         Person alice = new PhDStudent("Alice", 26, "Comp. Science", "AI");
5         Animal murka = new Cat("Murka", 5);
6         john.assignPet(murka); // John owns Rex
7         PersonRegistry registry = new PersonRegistry();
8         registry.addPerson(john);
9         registry.addPerson(alice);
10        // John goes on vacation and leaves Rex with Alice
11        john.leavePetWith(alice);
12        // Registry reflects that Alice is taking care of Rex
13        System.out.println(registry);
14        // John returns from vacation and retrieves Rex
15        john.retrievePetFrom(alice);
16        // Registry reflects that John has his dog back
17        System.out.println(registry);
18    }
19 }

```

Abstract class Person:

Every instance of the Person class have name, age, and possibly a pet (Animal), and *methods* like below:

- `assignPet (Animal pet)`: Assigns a pet to a person.
- `removePet ()`: Removes the pet from a person.
- `hasPet ()`: Checks if the person has a pet.
- `getOccupation ()`: Describes the person's occupation (to be implemented by subclasses).
- Additional methods like `toString ()`, `equals ()`, `hashCode ()`.

Classes Employee, Student, PhDStudent:

These extend Person, each with its specific fields and implementing `getOccupation ()`. Each subclass should have at least 1 new field different from its parent. The PhD students are

so consumed with their research that they can't have high-maintenance pets like dogs, they are too busy to provide constant attention. PhDStudent can have more independent pets like Cat, Fish, and Bird. The system should not allow PhDStudent to have a Dog; other people can't leave their dogs to PhDStudent as well.

Abstract class Animal:

Every animal have name and age, and each specific type of animal (e.g., Cat, Dog, Bird, Fish) has its way of making a sound via the `getSound()` method (to be implemented by subclasses).

Subclasses of Animal:

Implement concrete animals like Cat, Dog, Bird, and Fish, providing for each its specific sound when `getSound()` is called.

Class PersonRegistry:

A registry to hold a collection of people and allow operations like adding/removing people, finding people with/without pets, and printing details about each person and their pets.

Logic of Vacation:

1. People can leave their pets with others while they are on vacation. Implement a way for a person to temporarily transfer their pet to another person.
2. The system should ensure that, when a person leaves their pet, the pet is removed from their care and assigned to the temporary caretaker. When the person returns, they should be able to retrieve their pet.
3. You'll need to consider how to handle cases where a person doesn't have a pet, but tries to leave one with a friend or a service.
4. Consider how to model the temporary transfer of a pet and its return after a vacation.